

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 3
Collection Framework



Introduction

The Collection Framework was introduced in java from 1.2. Prior to java 1.2 there were independent classes for the various implementations of the Data Structures. The Collection Framework was introduced with the purpose of standardizing and having interoperability between various Data Structures. The framework relies on a standard set of interfaces which may be implemented in different ways by the actual data structure implementation classes.

Introduction

The interface have methods to allow for interoperation between the various data structure implementation classes. Here the data structure is used to manage instances and not for managing primitive data types.

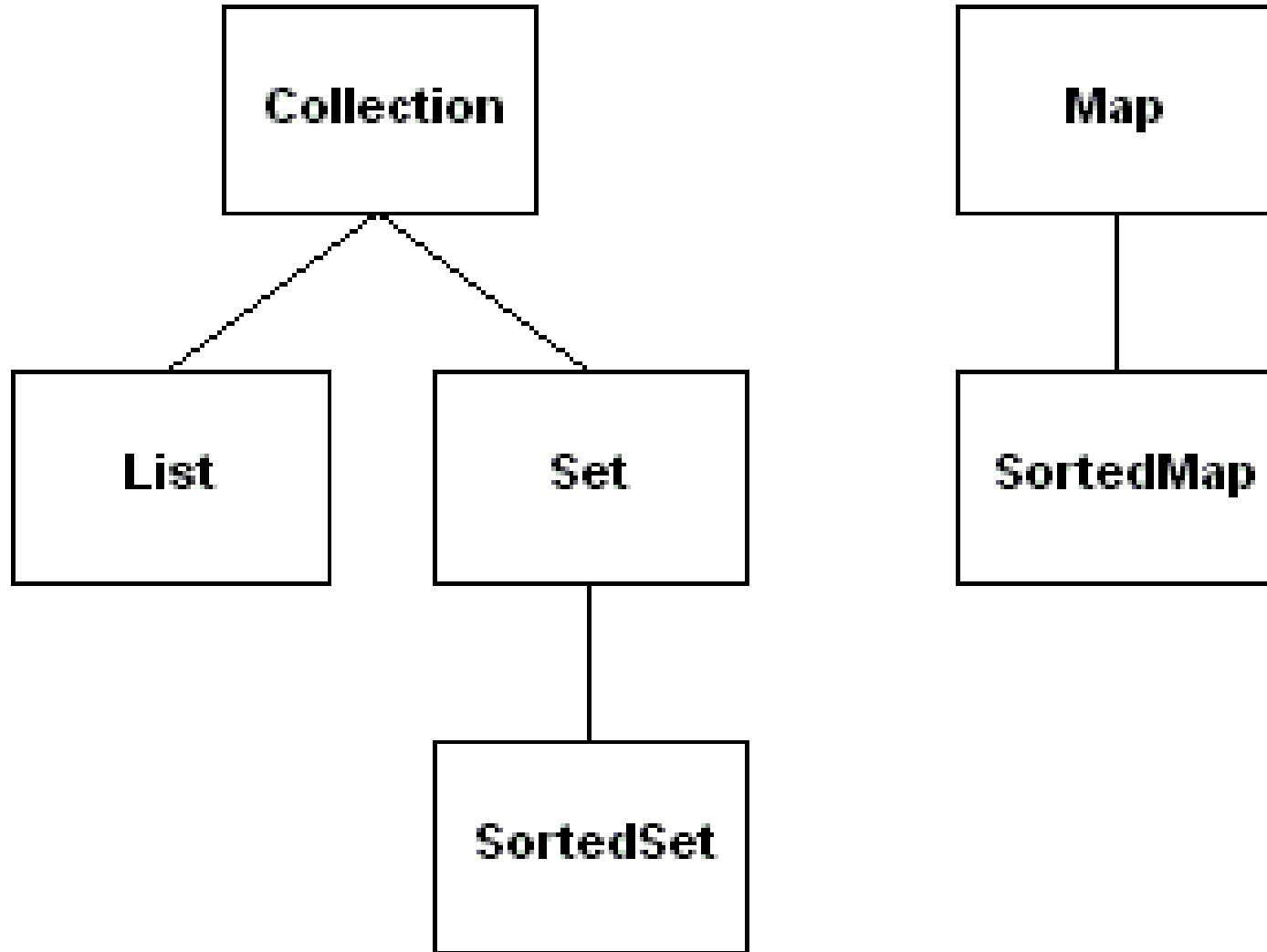
Remember that : from java 5.0 onwards we can also use primitive data types in collection framework through the feature boxing and unboxing conversions.

Collection Interface

The base interface is the Collection interface which represents any kind of collection of Objects. The methods in this interface define the kind of operations which may be implemented by the various data structure implementations classes.

Various operations that can be performed on collections are categorized into (1) basic operations, (2) array operations and (3) bulk operations.

Collection Interface



Collection Interface

Basic operations of the Collection interface are as follows :

Method	Usage
<code>boolean add(Object o)</code>	Useful to add a new object into the collection
<code>boolean remove(Object o)</code>	Remove s the specified object from the collection
<code>int size()</code>	Returns the number of elements in the collection
<code>boolean contains(Object o)</code>	Returns true if the object exist in the collection otherwise returns false
<code>boolean isEmpty()</code>	Returns true if the collection is empty otherwise returns false
<code>Iterator iterator()</code>	This method returns an instance of Iterator interface. An instance of the Iterator always has an underlying collection.

Collection Interface

Various methods of Iterator interface are as follows :

Method	Usage
boolean hasNext()	Returns true if the iterator has next element in the collection. Otherwise returns false.
Object next()	Returns next object from the collection.
void remove()	Removes the last element retrieved from the collection.

Collection Interface

e.g.

```
Collection c = new ..... ();  
Iterator iter = c.iterator();  
while (iter.hasNext())  
{  
    .....  
    x = iter.next();  
    if(.....)  
    {  
        iter.remove();  
    }  
}
```


Collection Interface

(2) We can perform array operation with the only one method available i.e. `Object[] toArray()` which returns an instance of the `Object[]`, whose size is the same as the size of the collection.

Collection Interface

(3) Bulk operations : We can perform various operations on the bulk data – for that various methods are as follows :

Method	Usage
boolean addAll(Collection c)	Add all the instances of given collection to the calling collection. Suppose we have two collections a and b then <code>a.addAll(b)</code> will result in $a = a \cup b$
boolean removeAll(Collection c)	Remove all the instances of given collection from the calling collection. Suppose we have two collections a and b then <code>a.removeAll(b)</code> will result in $a = a - b$
boolean retainAll(Collection c)	It will result in $a = a \cap b$
boolean containsAll(Collection c)	It will check for b is a subset of a

List Interface

The `java.util.List` interface is a subtype of the `java.util.Collection` interface. A List is an ordered Collection (sometimes called a sequence). meaning you can access the elements of a List in a specific order, and by an index too. List interface indicates the behavior of the collection of objects. It allows duplicate objects and one or more elements to be null. The user of this interface has precise control over where in the list each element is inserted. The user can access elements by integer index (position in the list), and search for elements in the list.

List Interface

There are two general-purpose List implementations in the Collections Framework:

(1) ArrayList

(2) LinkedList

These are the two general-purpose List implementations of the List interface.

ArrayList Interface

ArrayList extends AbstractList and implements List interface, Cloneable, Serializable. ArrayList capacity grows automatically. The ArrayList is not synchronized (means does not support multithreading). It permits all elements including null. ArrayList provides methods to manipulate the size of the array that is used internally to store the list. Array lists are created with an initial size. When this size is exceeded, the collection is automatically enlarged.

ArrayList Interface

When objects are removed, the array may be shrunk.

ArrayList has the constructors shown here:

ArrayList() - Constructs an empty list with an initial capacity of ten.

ArrayList(Collection c) - Constructs a list containing the elements of the specified collection, in the order they are returned by the collection's iterator.

ArrayList(int capacity) - Constructs an empty list with the specified initial capacity.

ArrayList Interface

Various methods of the interface are as follows :

Method	Usage
<code>void add(int index, Object o)</code>	Adds the specific object to the specified index
<code>void add(Object o)</code>	Adds the element at the end of the list
<code>void clear()</code>	Removes all the elements from the list
<code>Object clone()</code>	Returns a copy of ArrayList object.
<code>boolean contains(Object o)</code>	Returns true if the ArrayList contains the Object otherwise returns false
<code>Object get(int index)</code>	Returns the object available at given index
<code>int indexOf(Object o)</code>	Search for the object for its first occurrence and return index of that
<code>boolean isEmpty()</code>	Returns true if the list is empty otherwise false
<code>int lastIndexOf(Object o)</code>	Search for the object from backward for its first occurrence and return index of that

ArrayList Interface

Various methods of the interface are as follows :

Method	Usage
Object remove(int index)	Removes the object at given index
void removeRange(int start, int end)	Removes from this List all of the elements whose index is between start (inclusive) and end (exclusive)
Object set(int index, Object o)	Replaces the element at the specified index position in this list with the element specified by o
int size()	Returns the size of the list

[ex\ex63.java](#)

LinkedList Interface

LinkedList implementation of the List interface. Implements all optional list operations, and permits all elements (including null). In addition to implementing the List interface, the LinkedList class provides uniformly named methods to get, remove and insert an element at the beginning and end of the list. These operations allow linked lists to be used as a stack, queue, or double-ended queue (deque).

LinkedList Interface

All of the stack/queue/deque operations could be easily recast in terms of the standard list operations. They're included here primarily for convenience, though they may run slightly faster than the equivalent List operations.

All of the operations perform as could be expected for a doubly-linked list. Operations that index into the list will traverse the list from the beginning or the end, whichever is closer to the specified index.

LinkedList Interface

Note that this implementation is not synchronized. If multiple threads access a list concurrently, and at least one of the threads modifies the list structurally, it *must* be synchronized externally. (A structural modification is any operation that adds or deletes one or more elements) This is typically accomplished by synchronizing on some object that naturally encapsulates the list.

LinkedList Interface

If no such object exists, the list should be "wrapped" using the `Collections.synchronizedList` method. This is best done at creation time, to prevent accidental unsynchronized access to the list:

```
List list = Collections.synchronizedList(new LinkedList(...));
```

This class has only one constructor which initialize the `LinkedList`.

LinkedList Interface

Various methods of the interface are as follows :

Method	Usage
<code>void add(int index, Object o)</code>	Adds the o at the specified index
<code>boolean add(Object o)</code>	Add the specific element o at the end of the list
<code>boolean addAll(Collection c)</code>	Adds all the elements of Collection c to the calling list.
<code>boolean addAll(int index, Collection c)</code>	Adds all the elements of Collection c to the calling list at specific index point
<code>void addFirst(Object o)</code>	Adds the object o at first position
<code>void addLast(Object o)</code>	Adds the object o at the last position
<code>void clear()</code>	Removes all the elements from the list
<code>boolean contains(Object o)</code>	Returns true if the list contains the given object o otherwise returns false
<code>Object get(int index)</code>	Return the element at given index

LinkedList Interface

Various methods of the interface are as follows :

Method	Usage
Object getFirst()	Returns the first element from the list
Object getLast()	Returns the last element from the list
int indexOf(Object o)	Returns the index of given Object o
int lastIndexOf(Object o)	Returns the last index of given Object o
Object remove(int index)	Removes the object at given index
boolean remove(Object o)	Removes the given object o
Object removeFirst()	Removes the first element from the list
Object removeLast()	Removes the last element from the list
Object set(int index, Object o)	Replace the object o at given index
int size()	Returns the size of the list

[ex\ex64.java](#)

Vector class

The Vector class implements a growable array of objects. Like an array, it contains components that can be accessed using an integer index. However, the size of a Vector can grow or shrink as needed to accommodate adding and removing items after the Vector has been created.

Each vector tries to optimize storage management by maintaining a capacity and a capacityIncrement.

Vector class

The capacity is always at least as large as the vector size; it is usually larger because as components are added to the vector, the vector's storage increases in chunks the size of `capacityIncrement`. An application can increase the capacity of a vector before inserting a large number of components; this reduces the amount of incremental reallocation.

Vector class

This class contains four constructors which are as follows :

- (1) `Vector()` **// blank constructor**
- (2) `Vector(Collection c)` **// create vector from elements of other collection**
- (3) `Vector(int initialCapacity)` **// Creates a vector with given initial capacity**
- (4) `Vector(int initialCapacity, int capacityIncrement)` **// creates a vector with initial capacity and incremental capacity**

Vector class

Various methods of the class are as follows :

Method	Usage
<code>void add(int index, Object o)</code>	Adds the object at given index
<code>void add(Object o)</code>	Adds the object at the end of the vector
<code>void addAll(Collection c)</code>	Adds all the elements of collection to vector at the end
<code>void addAll(int index, Collection c)</code>	Adds all the elements of collection to vector at given index
<code>void addElement(Object o)</code>	Adds the object at the end of the vector
<code>int capacity()</code>	Returns the current capacity of the vector
<code>void clear()</code>	Removes all the elements from the vector
<code>boolean contains(Object o)</code>	Checks for the object contained in the vector and returns true if found otherwise return false
<code>Object elementAt(int index)</code>	Returns the object at given index
<code>Enumeration elements()</code>	Returns all elements in enumeration

Vector class

Various methods of the class are as follows :

Method	Usage
Object firstElement()	Returns the first element from the vector
Object get(int index)	Returns the object from the given index
int hashCode()	Returns the hashCode value of the given vector
int indexOf(Object o)	Returns the index of given object
int indexOf(Object o, int index)	Returns the first occurrence of given object after given index
void insertElementAt(Object o, int index)	Insert given object at given index
boolean isEmpty()	Returns true if the vector is empty otherwise returns false
Object lastElement()	Returns the last element from the vector
int lastIndexOf(Object o)	Returns the last occurrence of given object
int lastIndexOf(Object o,int index)	Search from backward for given object from given index

Vector class

Various methods of the class are as follows :

Method	Usage
Object remove(int index)	Removes the given index object
boolean remove(Object o)	Removes the given object
void removeAllElements()	Removes all the elements from the vector
void removeElementAt(int index)	Removes the element from given index
void removeRange(int from, int to)	Removes all element from given from and upto to where from inclusive while to exclusive
Object set(int index, Object o)	Replace the given object at given index
int size()	Returns the size of the vector

[ex\ex65.java](#)

Map interface

A Map is an object that maps keys to values. It is not an extension of the **collection** interface rather it has own interface hierarchy. Map provides a more general way for storing elements without containing duplicate keys. It allows you to store pairs of elements, termed "**keys**" and "**values**", where each key maps to one value. Thus the keys in a map must be unique.

The Collections Framework provides three general-purpose **Map** implementation:

Map interface

- (1) HashMap
- (2) TreeMap
- (3) LinkedHashMap

We will see about all these maps in detail but first we see the methods defined by the map interface in detail and then we will see about all this.

Map interface

Various methods of the interface are as follows :

Method	Usage
<code>void clear()</code>	Clear all the mappings from the map
<code>boolean containsKey(Object key)</code>	Returns true if the map contains the given key pair in the map
<code>boolean containsValue(Object value)</code>	Returns true if the map contains the given value with single or multiple keys in the map
<code>Object get(Object key)</code>	Returns the object mapped with given key
<code>boolean isEmpty()</code>	Returns true if the map is empty
<code>int size()</code>	Returns the size of the map

HashMap class

HashMap class is part of java.util package. HashMap class can add key and value put(key, value) pair elements. This HashMap permits null key and value. But HashMap is unsynchronized. HashMap class gives no guarantee to return as original order as entered.

Remember that : this class is more or less same as HashTable (See in next section) with the difference that it allows null values and it is unsynchronised.

HashMap class

Various methods of the class are as follows :

Method	Usage
<code>void clear()</code>	Removes all mappings from the map
<code>boolean containsKey(Object key)</code>	Returns true if the given key is there in map as key otherwise returns false
<code>boolean containsValue(Object value)</code>	Returns true if the given value is there in map as a value otherwise returns false
<code>Object get(Object key)</code>	Returns the object connected with given key
<code>boolean isEmpty()</code>	Returns true if the map is empty otherwise returns false
<code>Object put(Object value, Object key)</code>	Inserts a new element in the map
<code>Object remove(Object key)</code>	Removes the object associated with given key
<code>int size()</code>	Returns the size of the map

[ex\ex66.java](#)

HashTable Class

This class implements a hashtable, which maps keys to values. Any non-null object can be used as a key or as a value.

To successfully store and retrieve objects from a hashtable, the objects used as keys must implement the hashCode method and the equals method.

HashTable Class

Various methods of the class are as follows :

Method	Usage
<code>void clear()</code>	Clears the hashtable
<code>boolean contains(Object value)</code>	Returns true if the hashtable contains the given object value
<code>boolean containsKey(Object key)</code>	Returns true if the given key is there in the hashtable otherwise returns false
<code>boolean containsValue(Object value)</code>	Returns true if the given value is there in the hashtable otherwise returns false
<code>boolean equals(Object o)</code>	Returns true if the calling object and given object are equal otherwise returns false
<code>boolean isEmpty()</code>	Returns true if the hashtable is empty otherwise returns false
<code>Enumeration keys()</code>	Returns an enumeration of keys.
<code>Object remove(Object key)</code>	Removes the element with given key value
<code>int size()</code>	Returns the size of the HashTable

HashTable Class

Various methods of the class are as follows :

Method	Usage
Object remove(Object key)	Removes the element with given key value
int size()	Returns the size of the HashTable

[ex\ex67.java](#)

TreeMap class

The `TreeMap` class implements the `Map` interface by using a tree. A `TreeMap` provides an efficient means of storing key/value pairs in sorted order, and allows rapid retrieval. You should note that, unlike a hash map, a tree map guarantees that its elements will be sorted in ascending key order. (key, value) pairs are ordered on the key. A `Map` is an object that maps keys to values. Also called an Associative Array or Dictionary.

TreeMap class

Various methods of the class are as follows :

Method	Usage
<code>void clear()</code>	Clears the treemap
<code>boolean containsKey(Object key)</code>	Returns true if the given key is there in the TreeMap otherwise returns false
<code>boolean containsValue(Object value)</code>	Returns true if the given value is there in the TreeMap otherwise returns false
<code>Object firstKey()</code>	Returns the first (lowest) key from the map
<code>Object get(Object key)</code>	Returns the object mapped with given key
<code>Object lastKey()</code>	Returns the last (highest) key from the map
<code>Object remove(Object key)</code>	Removes the object for given key
<code>int size()</code>	Returns the size of the map

[ex\ex68.java](#)

Set interface

A collection that contains no duplicate elements. More formally, sets contain no pair of elements $e1$ and $e2$ such that $e1.equals(e2)$, and at most one null element. As implied by its name, this interface models the mathematical set abstraction.

The Set interface contains only methods inherited from Collection.

The Collection Framework provides two concrete set implementations:

1- [HashSet](#)

2- [TreeSet](#)

Set interface

Various methods of the interface are as follows :

Method	Usage
<code>boolean add(Object o)</code>	Adds a new object to the set
<code>void clear()</code>	Removes all elements from the set
<code>boolean contains(Object o)</code>	Returns true if the object is there in the set otherwise returns false
<code>int hashCode()</code>	Returns the hashCode for the set
<code>boolean isEmpty()</code>	Returns true if the set is empty otherwise returns false
<code>boolean remove(Object o)</code>	Removes the object from the set
<code>int size()</code>	Returns the size of the set
<code>Object [] toArray()</code>	Returns an array containing all the elements of set

HashSet Class

This class implements the Set interface and extends AbstractSet. It creates a collection that uses a hash table for storage. Hash table stores information by using a mechanism called hashing.

In hashing, the informational content of a key is used to determine a unique value, called its hash code. The hash code is then used as an index at which the data associated with the key is stored.

HashSet Class

Various methods of the class are as follows :

Method	Usage
boolean add(Object o)	Adds the given object to the HashSet
void clear()	Removes all the elements from the HashSet
boolean contains(Object o)	Returns true if the given object is there in the HashSet otherwise returns false
boolean isEmpty()	Returns true if the HashSet is empty otherwise returns false
boolean remove(Object o)	Removes the given object from the HashSet
int size()	Returns the size of HashSet

[ex\ex69.java](#)

TreeSet Class

This class implements the Set interface, The TreeSet implementations useful when you need to extract elements from a collection in a sorted manner. TreeSet stores objects in a sorted manner. TreeSet stores its elements in a tree and they are automatically arranged in a sorted order. TreeSet is not synchronized. If more than one thread wants to access it at the same time then it must be synchronized externally.

TreeSet Class

Various methods of the class are as follows :

Method	Usage
boolean add(Object o)	Adds the given object to the TreeSet
void clear()	Removes all the elements from the TreeSet
boolean contains(Object o)	Returns true if the object is there in the TreeSet otherwise returns false
Object first()	Returns the first (lowest) element from the set
boolean isEmpty()	Returns true if the set is empty otherwise returns false
Object last()	Returns the last (highest) element from the set
boolean remove(Object o)	Removes the given object from the set
int size()	Returns the size of the set

[ex\ex70.java](#)

Queue Interface

Queue interface is in `java.util` package of `java`. Queue is a linear collection that supports element insertion and removal at both ends. A Queue interface extends the Collection interface to define an ordered collection for holding elements in a FIFO (first-in-first-out) manner to process them i.e. in a FIFO queue the element that is inserted firstly will also get removed first. Queue does not require any fix dimension like String array and int array. Besides operations, queues also provide operations like insertion, removal, and inspection .

Queue Interface

Various methods of the interface are as follows :

Method	Usage
boolean add(Object o)	Adds the new object to the queue
Object element()	Returns the first element from the queue but not remove it from the queue
boolean offer(Object o)	Adds the new object to the queue
Object peek()	This method returns the first element from the queue but not remove it from the queue
Object poll()	Returns the first element from the queue and also remove from the queue
Object remove()	Returns the first element from the queue and also remove from the queue

[ex\ex71.java](#)

SortedSet Interface

SortedSet interface is part of java.util package. SortedSet interface can add value add(value) elements. Value of SortedSet can get by Iterator Interface. The java.util.SortedSet interface is a subtype of the java.util.Set interface. It behaves like a normal set with the exception that the elements are sorted internally. This means that when you iterate the elements of a SortedSet the elements are returned in the sorted order.

SortedSet Interface

Various methods of the interface are as follows :

Method	Usage
Object first()	Returns the first (lowest) element from the sorted set
Object last()	Returns the last (highest) element from the sorted set
SortedSet subSet(Object from, Object to)	Returns a new SortedSet in which objects are from (inclusive) to to(exclusive)
SortedSet tailSet(Object from)	Returns a new SortedSet in which objects are greater than or equal to the given from element.

[ex\ex72.java](#)

SortedMap Interface

The `java.util.SortedMap` interface is a subtype of the `java.util.Map` interface, with the addition that the elements stored in the map are sorted internally. The `SortedMap` interface extends `Map`. It ensures that the entries are maintained in ascending key order. By default the elements are iterated in ascending order, starting with the "smallest" and moving towards the "largest". But it is also possible to iterate the elements in descending order using the method `TreeMap.descendingKeySet()`.

SortedMap Interface

Various methods of the interface are as follows :

Method	Usage
Object firstKey()	Returns the first (lowest) element from the SortedMap
SortedMap headMap(Object toKey)	Returns a new SortedMap whose values are strictly less than the given key value.
Object lastKey()	Returns the last (highest) element from the SortedMap
SortedMap subMap(Object from, Object to)	Returns a new SortedMap of given values from(including) to to(excluding)
SortedMap tailMap(Object fromKey)	Returns a new SortedMap whose values are greater than or equal to given key

[ex\ex73.java](#)

More Details on Collection Framework

If you need more details on collection framework then you can visit the website

<http://www.devmanuals.com/tutorials/java/collections/>

On this website you will get all the details and examples of collection framework



Thank You

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 3
Lambda Expression



Introduction

Now you are ready to learn about lambda expressions, the most exciting change to the Java language in many years.

Need for Lambda

Annotations are like meta data, means you are free to add your code and can also apply them to variables, parameters, fields type declarations, methods and constructors.

```
class Worker implements ActionListener
{
    public void actionPerformed(ActionEvent event)
    {
        // do some work
    }
}
```

Need for Lambda

Then, when you want to repeatedly execute this code, you construct an instance of the Worker class.

The key point is that the `actionPerformed` method contains code that you want to execute later.

Or consider sorting with a custom comparator. If you want to sort strings by length instead of the default dictionary order, you can pass a `Comparator` object to the `sort` method:

Need for Lambda

```
class LengthComparator implements Comparator<String>
{
    public int compare(String first, String second)
    {
        return first.length() - second.length();
    }
}

...
Arrays.sort(strings, new LengthComparator());
```

Need for Lambda

The compare method isn't called right away. Instead, the sort method keeps calling the compare method, rearranging the elements if they are out of order, until the array is sorted. You give the sort method a snippet of code needed to compare elements, and that code is integrated into the rest of the sorting logic, which you'd probably not care to reimplement.

Both examples have something in common. A block of code was passed to someone—a timer, or a sort method. That code block was called at some later time.

The Syntax

Consider again the sorting example from the preceding section. We pass code that checks whether one string is shorter than another. We compute

```
first.length() - second.length()
```

What are first and second? They are both strings. Java is a strongly typed language, and we must specify that as well:

```
(String first, String second)  
-> first.length() - second.length()
```

The Name

Why the name? Many years ago, before there were any computers, the logician Alonzo Church wanted to formalize what it means for a mathematical function to be effectively computable. (Curiously, there are functions that are known to exist, but nobody knows how to compute their values.) He used the Greek letter lambda (λ) to mark parameters. Had he known about the Java API, he would have written

```
 $\lambda$ first. $\lambda$ second.first.length() - second.length()
```

Usage

You have just seen one form of lambda expressions in Java: parameters, the `->` arrow, and an expression. If the code carries out a computation that doesn't fit in a single expression, write it exactly like you would have written a method: enclosed in `{}` and with explicit return statements. For example,

```
(String first, String second) ->
{
    if (first.length() < second.length()) return -1;
    else if (first.length() > second.length()) return 1;
    else return 0;
}
```

Usage

If a lambda expression has no parameters, you still supply empty parentheses, just as with a parameterless method:

```
() -> { for (int i = 100; i >= 0; i--) System.out.println(i); }
```

[ex\lambda\Java8Tester.java](#)

[ex\lambda\Java8Test.java](#)

Functional Interfaces

As we discussed, there are many existing interfaces in Java that encapsulate blocks of code, such as `ActionListener` or `Comparator`. Lambdas are compatible with these interfaces.

You can supply a lambda expression whenever an object of an interface with a single abstract method is expected. Such an interface is called a functional interface.

Functional Interfaces

To demonstrate the conversion to a functional interface, consider the `Arrays.sort` method. Its second parameter requires an instance of `Comparator`, an interface with a single method. Simply supply a lambda:

```
Arrays.sort(words,  
(first, second) -> first.length() - second.length());
```


Functional Interfaces

Behind the scenes, the `Arrays.sort` method receives an object of some class that implements `Comparator<String>`.

Invoking the `compare` method on that object executes the body of the lambda expression. The management of these objects and classes is completely implementation dependent, and it can be much more efficient than using traditional inner classes. It is best to think of a lambda expression as a function, not an object, and to accept that it can be passed to a functional interface.

Method References

Sometimes, there is already a method that carries out exactly the action that you'd like to pass on to some other code. For example, suppose you simply want to print the event object whenever a timer event occurs. Of course, you could call

```
Timer t = new Timer(1000, event -> System.out.println(event));
```

Method References

It would be nicer if you could just pass the `println` method to the `Timer` constructor.

Here is how you do that:

```
Timer t = new Timer(1000, System.out::println);
```

The expression `System.out::println` is a *method reference* that is equivalent to the lambda expression `x -> System.out.println(x)`.

Method References

As another example, suppose you want to sort strings regardless of letter case. You can pass this method expression:

```
Arrays.sort(strings, String::compareToIgnoreCase)
```

As you can see from these examples, the `::` operator separates the method name from the name of an object or class. There are three principal cases:

Method References

- *object::instanceMethod*
- *Class::staticMethod*
- *Class::instanceMethod*

In the first two cases, the method reference is equivalent to a lambda expression that supplies the parameters of the method. As already mentioned,

`System.out::println` is equivalent to
`x -> System.out.println(x)`.

Similarly,

Method References

`Math::pow`

is equivalent to

`(x, y) -> Math.pow(x, y).`

In the third case, the first parameter becomes the target of the method. For example,

`String::compareToIgnoreCase`

is the same as

`(x, y) -> x.compareToIgnoreCase(y).`

Constructor References

Constructor references are just like method references, except that the name of the method is new.

For example, `Person::new` is a reference to a `Person` constructor.

Which constructor? It depends on the context. Suppose you have a list of strings. Then you can turn it into an array of `Person` objects, by calling the constructor on each of the strings, with the following invocation:

Constructor References

```
ArrayList<String> names = . . . ;  
Stream<Person> stream =  
names.stream().map(Person::new);  
List<Person> people =  
stream.collect(Collectors.toList());
```




Thank You

Generics Fundamentals

Generics introduce type parameters into Java, allowing you to define classes, interfaces, and methods where the data type is specified later.

Theory: Why We Use Them

The primary goals of Generics are:

1. **Type Safety:** Catches invalid type assignments at compile-time(when you write the code), preventing errors like `ClassCastException` at runtime.
2. **Code Reusability:** A single class or method can work with multiple data types without duplication.
3. **No Casting:** Eliminates the need to manually cast objects retrieved from collections, making code cleaner and safer.

Generic Class

A Generic Class is defined with one or more type parameters (like `T`) enclosed in angle brackets (`<>`) after the class name. This parameter acts as a placeholder for the actual data type.

Theory: The Placeholder

When you define a class like `Box<T>`, you are essentially saying, “This `Box` can hold any type, and whatever type that is, we’ll call it `T` inside this class definition.” When you create an object like `new Box<String>()`, the compiler replaces every instance of `T` with `String` for that specific object.

Example: Holder Class

This class holds a single item of a specified type `T`.

```
class Holder {
    private T item;

    public Holder(T item) {
        this.item = item;
    }

    public T getItem() {
        return item;
    }
}

public class MinimalGenericClass {
    public static void main(String[] args) {
```

```

Holder numHolder = new Holder<>(100);
int myNum = numHolder.getItem();
}
}

```

Generic Methods

A Generic Method introduces its own type parameter(s). The type parameter is declared ****before**** the method's return type.

Theory: Localized Flexibility

Generic methods are useful when you want the flexibility of generics for just **one** specific method, perhaps within a non-generic class. The compiler infers the actual type of T based on the arguments you pass to the method.

Example: `getMiddle` Method

A single static method, `getMiddle`, can handle any array type.

```

public class MinimalGenericMethod {
    public static T getMiddle(T[] array) {
        return array[array.length / 2];
    }

    public static void main(String[] args) {
        Integer[] numbers = { 1, 5, 9 };
        String[] words = { "Apple", "Banana", "Cherry" };
        System.out.println(getMiddle(numbers)); // Output: 5
        System.out.println(getMiddle(words)); // Output: Banana
    }
}

```

A Generic Class with Multiple Type Parameters

A generic class can accept multiple type arguments, separated by commas (e.g., `<K, V>`).

Theory: Structuring Complex Data

When a class needs to manage two different but related data types—such as a key and its associated value (like in a `Map` entry)—multiple parameters are essential to maintain strong type checking for both parts.

Example: `KeyValue` Class

This class holds a key of type `K` and a value of type `V`.

```
class KeyValue<K, V> {
    private K key;
    private V value;
    public KeyValue(K key, V value) {
        this.key = key;
        this.value = value;
    }
    public K getKey() { return key; }
    public V getValue() { return value; }
}

public class MinimalMultipleTypes {
    public static void main(String[] args) {
        KeyValue<String, Integer> studentId = new KeyValue<>("Alex", 1001);

        System.out.println(studentId.getKey() + " ID: " + studentId.getValue());
    }
}
```

****Bounded Types with Generics****

Bounded Types restrict the types that can be substituted for a type parameter. This is done

\### Theory: Enabling Method Calls

If you define a generic method that needs to call a method specific to the `Number` class (I

 Syntax: `<T extends SuperClassOrInterface>`

Example: `getDoubleValue` Method

This method is restricted to only work with types that extend `Number`.

```
public class MinimalBoundedTypes {

    &nbsp; public static <T extends Number> double getDoubleValue(T num) {
    &nbsp;         return num.doubleValue();
    &nbsp; }

    &nbsp; public static void main(String\[] args) {
    &nbsp;         Integer i = 15;
    &nbsp;         Double d = 3.14;

    &nbsp;         System.out.println(getDoubleValue(i)); // Output: 15.0
    &nbsp;
    &nbsp;         // String s = "Text";
```

```

    &nbsp; // getDoubleValue(s); // COMPILE-TIME ERROR!

    &nbsp; }

}

```

Rules for Generic Types

These rules govern how generics can and cannot be used, primarily due to the underlying mechanism known as Type Erasure.

1. Primitives are Not Allowed:

Rule: You must use ****Wrapper Classes**** (e.g., `Integer`, `Double`) instead of primitive types (e.g., `int`, `double`) as type arguments.

Bad: `List<int>`

Good: `List<Integer>`

2. Cannot Instantiate Type Parameters:

Rule: You cannot create an object using the generic type parameter, e.g., `new T()`.

Reason: The compiler doesn't know what type `T` will be at runtime, so it cannot choose the correct constructor.

3. No Static Fields of Type Parameter Type:

Rule: You cannot declare a static variable using the generic type parameter.

Reason: Static fields are shared across all instances. A static field in `Box<String>` would be the same as in `Box<Integer>`, which would violate type safety.

4. Type Erasure:

Rule: Generics are primarily a compile-time feature. Once the code is compiled, the type parameters are erased and replaced with their bounds (or `Object` if no bounds exist).

Impact: This is why you cannot use generic types with `instanceof` (e.g., `if (obj instanceof Box<Integer>)`). The type information (`<Integer>`) is gone at runtime.

Processing Lambda Expressions

Lambda expressions are essentially anonymous functions that are treated as objects in Java, enabled by the Functional Interface and the `invokedynamic` instruction.

1. The Contract: Functional Interface (FI)

A Lambda expression must target a Functional Interface—an interface with exactly one abstract method (SAM).

The FI's method signature dictates the required parameters and return type of the Lambda.

2. The Compiler's Job

The compiler **does not** generate a new `.class` file for the Lambda.

It places the Lambda's code into a private, synthetic method within the containing class (the "Lambda Body").

The actual call site is replaced with the special `invokedynamic` bytecode instruction. This defers the final connection (linkage) to runtime.

3. The JVM's Job (Runtime Bootstrap)

The first time the JVM encounters the `invokedynamic` instruction, it calls a bootstrap method (specifically `Metafactory.metafactory`).

The bootstrap method dynamically generates a class at runtime that implements the target Functional Interface.

This generated class's single abstract method simply calls (delegates to) the private Lambda Body method created by the compiler.

The generated link is then cached for all subsequent calls, making the execution efficient.

Lambda expressions are processed by the compiler which creates a private synthetic method for the body and replaces the call with the `invokedynamic` instruction, which the JVM uses at runtime to dynamically generate an object implementing the Functional Interface and link it to the call site.

Variable Scope in Java Theory

Variable scope defines the region of the program where a variable is accessible and valid. It determines the **lifetime** of a variable (when memory is allocated and released).

In Java, there are three primary levels of scope:

1. Class/Static Scope (Field)
2. Instance Scope (Field)
3. Method/Local Scope (Variable)
- 1\ . Class (Static) Scope

Theory

Definition: Variables declared using the **static** keyword. They are also called static fields or class variables.

Lifetime: They are created when the class is loaded into memory and are destroyed when the program terminates.

Accessibility: Accessible from anywhere within the class and can be accessed outside the class using the class name (e.g., **ClassName.variableName**).

Memory: Only a single copy of the variable exists for the *entire class*, regardless of how many objects (instances) are created.

Example

```
class Counter {

    public static int totalObjects = 0;  (// Static Scope: Shared by ALL objects
of the Counter class)

    public Counter() {
        totalObjects++; (// Incremented every time an object is created)
    }
}

public class StaticScopeExample {

    public static void main(String[] args) {

        System.out.println("Start:" + Counter.totalObjects);( // Output: 0 )( //
Accessing the static variable using the class name)

        Counter c1 = new Counter();

        Counter c2 = new Counter();

        System.out.println("End:" + Counter.totalObjects); (// Output: 2) ( // The
single static variable reflects the change from BOTH objects)

    }
}
```

2\. Instance Scope

Theory

Definition: Variables declared directly within a class but ****outside**** any method or block, and without the **static** keyword. They are also called instance fields or non-static fields.

Lifetime: They are created when an object (instance) of the class is created using the **new** keyword and are destroyed when the object is garbage-collected.

Accessibility: Accessible by methods within the same object. They must be accessed through an object reference (e.g., **objectName.variableName**).

Memory: Each object gets its own copy of the variable.

Example

```
class Dog {  
  
    public String name; ( // Instance Scope: Each Dog object has its own separate  
    copy of 'name')  
  
    public Dog(String name) {  
        this.name = name;  
    }  
}  
  
public class InstanceScopeExample {  
    public static void main(String[] args) {  
        Dog d1 = new Dog("Max");  
        Dog d2 = new Dog("Luna"); (// d1.name and d2.name hold different values  
        (separate copies))  
        System.out.println("Dog 1:" + d1.name); (// Output: Max)  
        System.out.println("Dog 2:" + d2.name); (// Output: Luna)  
    }  
}
```

3\ . Method/Local Scope

Definition: Variables declared inside a method, constructor, or any block of code (like an **if** statement or **for** loop).

Lifetime: They are created when the block of code is entered and are destroyed as soon as the block is exited.

Accessibility: Only accessible from their declaration point until the end of the block in which they are declared. They are not visible outside of that block.

Initialization: Unlike fields, local variables are not given a default value by Java and must be explicitly initialized before use.

Example

```
public class LocalScopeExample {  
    public static void main(String[] args) {  
        int x = 10; (// x has Method Scope (main method))  
  
        if (x > 5) {  
  
            String y = "High Value"; (// y has Local/Block Scope (only within this if  
block))  
            System.out.println("Inside:" + y); (// Output: Inside: High Value)  
        }  
  
        (// System.out.println(y); // COMPILE-TIME ERROR: y is out of scope!)  
  
        System.out.println("Outside:" + x); (// Output: Outside: 10 (x is still in  
scope))  
    }  
}
```